P1811R0
Relaxing redefinition restrictions for re-exportation robustness

Published Proposal, 2019-07-18

This version:
http://wg21.link/p1811r0

Authors:
Richard Smith (Google)
Gabriel Dos Reis (Microsoft)

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The current redefinition rules for entities attached to the global module unnecessarily disallows some natural code patterns. We propose to relax these rules to permit the relevant code patterns.

Table of Contents

§ 1. Background

In C++ code prior to C++20, interfaces are exposed by textual inclusion. This results in definitions of the same entity appearing in multiple translation units. We require (though unfortunately do not consistently enforce in practice) that all such definitions are "the same". ("The same" is a complex issue, but the details aren't salient to this paper.)

In addition, we require that there is at most one definition of each entity in each translation unit. From an implementation perspective, this (necessarily) weak enforcement of the One Definition Rule serves a number of purposes, notably:

- it permits simpler implementations, as there is no need to cope with encountering a definition of an entity when another definition is already known
- it ensures that compilation time is not wasted processing redundant redefinitions

> EXAMPLE 1
>
> ```
> // foo.h -- user forgot the include guards
> struct X { int foo; };
> ```
>
> ```
> #include "foo.h"
> #include "foo.h" // error, redefinition
> ```

- it catches some bugs where there is an accidental name collision between two different entities of the same kind (eg, two different `struct X`s)

EXAMPLE 2

```
// foo.h
struct X { int foo; };
```

```
// bar.h
struct X { int bar; };
```

```
#include "foo.h"
#include "bar.h" // error, redefinition
```

However, it is important to note that the requirement of only one definition per translation unit is not necessary for the soundness of the language: the requirement that all definitions are "the same" is sufficient. We could accept example 1 and consider example 2 to be ill-formed (likely with no diagnostic required), but it is preferable to reject such redefinition cases when there is not a compelling reason to permit them.

In the current C++20 draft, we continue to allow multiple definitions of entities that are attached to the global module (entities attached to named modules can only have one definition). We now allow definitions to be made reachable from translation units other than the one in which they are defined (via various forms of `import`), and so in an attempt to preserve the above useful properties to the extent possible, we disallow redefinition of an entity if a prior definition is "necessarily reachable" -- which, for names attached to the global module, means the definition can be found through a path of zero or more `import` declarations, where each one other than the first is exported.

Note that this necessary reachability rule does not satisfy the first goal above: implementations do need to cope with encountering a definition of an entity when another definition is already known, because there are cases where such a definition must be available to template instantiation in some contexts, despite not being necessarily reachable. (In such cases, we might describe the definition as "incidentally reachable", although the standard wording doesn’t use that term.)

EXAMPLE 3

```
// foo.h
#ifndef FOO_H
#define FOO_H
struct Foo {
  constexpr int get() const { return 42; }
};
#endif
```

```
// bar.h
#include "foo.h"
// ...
```

```
export module M;
import "bar.h";

export template<typename T> constexpr T make() {
  return T(Foo().get());
}
```

```
import M;

// OK, array bound is 42
// Note that make<int> can be instantiated and used here, even though Fo
// not necessarily reachable from this context.
int arr[make<int>()];

// OK, definition of Foo is valid, because prior definition is not neces
// reachable in this context. (Note that the include guard macro FOO_H i
// made visible by the import of M.)
#include "foo.h"
```

§ 2. Problem

We permit entities attached to the global module to be made reachable through a named module, in one of several ways:

```
module;
#include "a.h" // struct A {};
export module M;
export import "b.h"; // struct B {}; (in importable header)
export A f();
```

Here, the definitions of `A` and `B` are reachable in an importer of module `M`:

- `A` is reachable implicitly because it's used in the declaration of `f`
- `B` is reachable explicitly because it's defined in an exported header unit

This re-export of entities attached to the global module divorces the entities from their include guard macros, which exposes the possibiltiy that a consumer of this module interface will encounter a redefinition error with the current rule:

```
import M;
#include "a.h" // error, redefinition of A
#include "b.h" // OK, but see below
```

Due to the `#include` translation rule, the `#include "b.h"` directive is translated to `import "b.h";` because `"b.h"` is an importable header, so there is no redefinition error in that case. But this does not remove all risk:

```
// b-impl.h, not an importable header
#ifndef B_IMPL_H
#define B_IMPL_H
struct B {};
#endif
```

```
// b.h, importable header
#include "b-impl.h"
```

```
import M;
#include "b-impl.h" // error, redefinition of B
```

In both cases, disassociating the `#include` guard macro from the definitions that it is guarding has removed the protection from redefinition errors that we were traditionally relying upon.

§ 3. Proposed solution

Change the redefinition restriction as follows: for entities attached to the global module, permit at most one definition per translation unit, regardless of whether any definition of the entity is already reachable.

What we gain:

- increased robustness in the presence of re-exportation of entities attached to the global module
- a slightly simpler rule (we can entirely remove the notion of "necessarily reachable" and replace it with "in the same translation unit"
- a rule that is in some sense closer to the pre-C++20 rule
- a lower burden on implementations, as there is no longer any requirement to precisely track which definitions are necessarily reachable

What we lose:

- in principle, there are some cases where we would previously have caught a name collision between two different definitions of the same type, and will no be guaranteed to give a diagnostic; as usual, if there are multiple definitions that are not "the same" (according to the ODR), the program is ill-formed with no diagnostic required

Note that this change should apply to all constructs that are permitted to be repeated across translation units but not within a single translation unit, including redeclarations of default arguments and default template arguments in addition to redefinitions of classes, functions, variables, and enumerations.

§ 3.1. Making `#include` translation optional

Assuming the main proposal is accepted, there is another question we can now consider: should we require implementations to perform `#include` translation for all headers for which they would permit an `import`?

Previously, we required include translation in part in order to mitigate the effects of the problem that is solved by this proposal. In the simple `b.h` example above, `#include` translation saves us from a redefinition error unless a definition in an imported header can also be found in a textual header.

With the acceptance of this proposal, we can reconsider whether we want to give implementations the freedom to choose whether to map `#include` directives to `import` declarations separately from choosing which header files are importable. At least one implementation vendor believes that migration to modules would be eased by permitting these two decisions to be decoupled.

We propose that implementations be permitted to choose not to perform `#include` translation for importable headers.

§ 4. Alternatives

If we keep the current redefinition rule, we will need to ensure that current header file techniques properly prevent redefinition. This would likely entail

- exporting macros from named modules whenever any entity attached to the global module is exported
- ensuring that export of entities from the global module has the same level of granularity as include guard macros (the export must be "all or nothing" because an include guard macro can’t be half-defined)

If we do not wish to allow macro export from named modules and want to keep the current rule for redefinitions, we conclude that we would need to disallow re-exportation of entities attached to the global module entirely.

We don’t consider any of the alternatives in this area to be acceptable, as they would require us to give up important properties (the ability for a module to export declarations from a non-modular library, or the guarantee that a named module import does not bring in macros).

§ 5. Implementation experience

This approach has been implemented in the Microsoft compiler for 3+ years.

This approach has also been implemented in a branch of the Clang compiler, and confirmed to fix the redefinition errors in the above examples.

§ 6. Wording

§ 6.1. Allow redefinition when a prior definition is reachable

Change in 6.2 [basic.def.odr] paragraph 1:

> No translation unit shall contain more than one definition of any ~~A~~ variable, function, class type, enumeration type, or template ~~shall not be defined where a prior definition is necessarily reachable (10.6); no diagnostic is required if the prior declaration is in another more than once in a single translation unit~~ .

Change in 6.2 [basic.def.odr] paragraph 12:

> There can be more than one definition of a class type (Clause 11), enumeration type (9.6), inline function with external linkage (9.1.6), inline variable with external linkage (9.1.6), class template (Clause 13), non-static function template (13.6.6), concept (13.6.8), static data member of a class template (13.6.1.3), member function of a class template (13.6.1.1), or template specialization for which some template parameters are not specified (13.8, 13.6.5) in a program provided that ~~no prior definition is necessarily reachable (10.6) at the point where a definition appears~~ each definition appears in a different translation unit , and provided the definitions satisfy the following requirements [...].

Drafting note: The two prior changes revert edits from P1103R3.

§ 6.2. Cleanup: remove now-unused definition of "necessarily reachable" for declarations

Change in 10.5 [module.context] paragraph 7:

> [*Example*: … so the definition of `X` ~~is not necessarily~~ need not be reachable, as described in [module.reach]. —*end example*]

Change in 10.6 [module.reach] paragraph 3:

> A declaration *D* is reachable ~~or necessarily reachable, respectively,~~ if, for any point *P* in the instantiation context (10.5),
>
> - *D* appears prior to *P* in the same translation unit, or
> - *D* is not discarded (10.4), appears in a translation unit that is reachable ~~or necessarily reachable~~ from *P* ~~, respectively,~~ and either does not appear within a *private-module-fragment* or appears in a *private-module-fragment* of the module containing *P*.

§ 6.3. Make `#include` translation optional

Change in 15.2 [cpp.include] paragraph 7:

> If the header identified by the *header-name* denotes an importable header (10.3), it is implementation-defined whether the preprocessing directive is instead replaced by the *preprocessing-token*s
>
> ```
> import header-name ;
> ```

§ 6.4. Feature test macro

While a feature test macro for this functionality is not obviously useful, a feature test macro for the Modules feature overall does seem useful.

Add a feature test macro `__cpp_modules` with a suitable value to Table 17 in 15.10 [cpp.predefined].